



学习笔记

作者：北斗逍遥-张嘉
时间：2025 年 10 月 23 日

目录

1 第一章：数学工具篇	1
1.1 分析篇	1
1.2 代数篇	1
1.3 概率论与数理统计篇	1
1.3.1 基础知识	1
1.3.2 随机过程	1
1.3.3 马尔可夫相关	1
1.3.4 贝叶斯优化	1
1.4 数论篇	1
1.5 实变函数篇	1
1.6 泛函分析篇	1
1.7 常微分方程篇	1
1.8 拓扑篇	1
2 第二章：理论分析证明框架汇总	2
2.1 零阶优化算法	2
2.1.1 相关概念	2
2.1.2 常见的假设	2
2.1.3 收敛性分析	2
2.2 内存分析	2
3 第三章：AI 代码整理	3
3.1 Pytorch 辅助性工具	3
3.1.1 日志管理	3
3.1.2 终端自动化调参	6
3.1.3 进度条设置	10
3.1.4 查询服务器显卡属性设置	11
3.1.5 wandb 学习	12
3.1.6 Github 项目管理	14
3.1.7 Tikz 的使用查询	14
3.2 训练模型的简易流程	16
3.2.1 基本流程	16
3.2.2 基本代码模块详解	17
3.2.3 定制代码模块详解	21
3.2.4 基础环境配置	21
3.3 研究领域和代码编写示例	23
3.3.1 图像识别 (Computer Vision)	23
3.3.2 自然语言处理 (Natural Language Processing)	23

3.3.3	生成式学习 (Generative Learning)	23
3.3.4	多模态 (Multimodal)	23
3.3.5	脉冲神经网络相关 (SNN)	23
4	第四章: AI 知识整理	24
5	第五章: 文章汇总整理	25
5.1	黑盒-零阶优化问题	25
5.2	大模型微调相关	25
5.3	SNN 相关 [校准]	25
5.4	SNN 相关 [量化]	25
5.5	SNN 相关 [马尔可夫]	25
5.6	SNN 相关 [编码]	25
6	第六章: 技术汇总整理	26
6.1	梯度优化	26
6.1.1	神经网络架构搜索	26
7	第七章: 项目练习	28
A	常见的的数据集汇总	30
A.1	图像数据	30
A.1.1	常见的 CycleGAN 数据:	30
A.2	文本数据	31
A.3	脉冲数据	31
A.4	音频数据	31
A.5	视频数据	31

Chapter 1

第一章：数学工具篇

§ 1.1 分析篇

§ 1.2 代数篇

§ 1.3 概率论与数理统计篇

1.3.1 基础知识

1.3.2 随机过程

1.3.3 马尔可夫相关

1.3.4 贝叶斯优化

§ 1.4 数论篇

§ 1.5 实变函数篇

§ 1.6 泛函分析篇

§ 1.7 常微分方程篇

§ 1.8 拓扑篇

Chapter 2

第二章：理论分析证明框架汇总

§2.1 零阶优化算法

2.1.1 相关概念

2.1.2 常见的假设

2.1.3 收敛性分析

§2.2 内存分析

Chapter 3

第三章：AI 代码整理

§3.1 Pytorch 辅助性工具

3.1.1 日志管理

在软件开发、系统运维等领域，logging 日志是一种非常重要的记录机制，其核心作用是追踪系统运行过程中的关键信息、行为和状态，以便于后续的调试、监控、分析和问题排查。

基本日志配置

基本示例如下：

```
1 import logging
2 import os
3 from datetime import datetime
4
5 def setup_logger(log_dir="logs"):
6     # 创建日志目录
7     if not os.path.exists(log_dir):
8         os.makedirs(log_dir)
9
10    # 日志文件名：包含时间戳
11    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
12    log_file = os.path.join(log_dir, f"training_{timestamp}.log")
13
14    # 配置日志
15    logging.basicConfig(
16        level=logging.INFO, # 日志级别: DEBUG, INFO, WARNING, ERROR, CRITICAL
17        format="%(asctime)s-%(name)s-%(levelname)s-%(message)s",
18        handlers=[
19            logging.FileHandler(log_file), # 输出到文件
20            logging.StreamHandler()       # 同时输出到控制台
21        ]
22    )
23
24    return logging.getLogger(__name__)
25
26 # 使用示例
```

```

27 logger = setup_logger()
28 logger.debug("这是调试信息")      # 不会显示，因为级别是INFO
29 logger.info("训练开始")           # 会显示
30 logger.warning("学习率可能过高")
31 logger.error("数据加载失败")

```

Listing 3.1: 基本日志配置

日志文件管理

基本示例如下：

```

1  import logging
2  from logging.handlers import RotatingFileHandler, TimedRotatingFileHandler
3  import os
4
5  def setup_advanced_logger(log_dir="logs"):
6      if not os.path.exists(log_dir):
7          os.makedirs(log_dir)
8
9      logger = logging.getLogger(__name__)
10     logger.setLevel(logging.INFO)
11
12     # 格式设置
13     formatter = logging.Formatter("%(asctime)s-%(name)s-%(levelname)s-%(message)s")
14
15     # 按大小轮转：每个文件最大10MB，保留5个备份
16     file_handler = RotatingFileHandler(
17         os.path.join(log_dir, "training.log"),
18         maxBytes=10*1024*1024, # 10MB
19         backupCount=5,
20         encoding="utf-8"
21     )
22
23     # 或者按时间轮转：每天一个日志文件
24     # file_handler = TimedRotatingFileHandler(
25     #     os.path.join(log_dir, "training.log"),
26     #     when="D", # 每天轮转
27     #     interval=1,
28     #     backupCount=7, # 保留7天
29     #     encoding="utf-8"
30     # )
31
32     file_handler.setFormatter(formatter)
33     logger.addHandler(file_handler)
34
35     # 同时输出到控制台
36     console_handler = logging.StreamHandler()
37     console_handler.setFormatter(formatter)
38     logger.addHandler(console_handler)
39
40     return logger
41

```



```

42 # 在PyTorch训练中的使用示例
43 def train_model(logger):
44     logger.info("开始模型训练")
45     for epoch in range(10):
46         logger.info(f"==== 第{epoch+1}轮训练====")
47         # 训练代码...
48         loss = 0.5 - epoch*0.05 # 示例损失值
49         accuracy = 0.6 + epoch*0.03 # 示例准确率
50         logger.info(f"损失:{loss:.4f}, 准确率:{accuracy:.4f}")
51
52     if loss < 0.2:
53         logger.warning("损失值过低, 可能存在过拟合风险")
54
55     logger.info("模型训练完成")

```

Listing 3.2: 日志文件的轮转

查看和解析日志文件

基本示例如下:

```

1 import re
2 import pandas as pd
3
4 def parse_log_file(log_path):
5     """解析日志文件, 提取训练指标"""
6     # 正则表达式匹配训练指标行
7     pattern = r"(\d{4}-\d{2}-\d{2}\d{2}:\d{2}:\d{2})-.*-INFO-损失:([\d.]+), 准确率:([\d.]+)"
8
9     data = []
10    with open(log_path, "r", encoding="utf-8") as f:
11        for line in f:
12            match = re.match(pattern, line.strip())
13            if match:
14                timestamp = match.group(1)
15                loss = float(match.group(2))
16                accuracy = float(match.group(3))
17                data.append({
18                    "timestamp": timestamp,
19                    "loss": loss,
20                    "accuracy": accuracy
21                })
22
23    return pd.DataFrame(data)
24
25 # 使用示例
26 # df = parse_log_file("logs/training.log")
27 # print(df)

```

Listing 3.3: 解析日志文件

将日志信息整理到表格

基本示例如下：

```

1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  def log_to_dataframe_and_visualize(log_path):
5      # 解析日志文件
6      df = parse_log_file(log_path)
7
8      if df.empty:
9          print("没有找到有效的训练指标数据")
10         return
11
12     # 保存为CSV文件
13     df.to_csv("training_metrics.csv", index=False)
14     print("训练指标已保存到training_metrics.csv")
15
16     # 可视化
17     plt.figure(figsize=(12, 5))
18
19     plt.subplot(1, 2, 1)
20     plt.plot(df["loss"])
21     plt.title("训练损失")
22     plt.xlabel("轮次")
23     plt.ylabel("损失值")
24
25     plt.subplot(1, 2, 2)
26     plt.plot(df["accuracy"])
27     plt.title("训练准确率")
28     plt.xlabel("轮次")
29     plt.ylabel("准确率")
30
31     plt.tight_layout()
32     plt.savefig("training_metrics.png")
33     plt.show()
34
35     return df
36
37     # 使用示例
38     # metrics_df = log_to_dataframe_and_visualize("logs/training.log")

```

Listing 3.4: 输出至表格

3.1.2 终端自动化调参

在深度学习（尤其是 PyTorch 训练）和日常开发中，shell 脚本（.sh 文件）是一种极其实用的工具，它的核心作用是自动化重复操作、批量执行命令、管理复杂 workflow。

shell-基础语法

一些常见的语法可供查阅

声明变量

```

1  # 声明一个变量
2  name="John"
3  # 声明变量并设置默认值
4  name=${name:-zj}
5  # 使用变量
6  echo "Hello, ${name}" # 输出: Hello, John!

```

Listing 3.5: 声明变量

if-else

```

1  age=20
2  # -eq: 等于, --ne: 不等于, -gt: 大于, -lt: 小于, -ge: 大于等于, -le: 小于等于
3  if [ $age -ge 18 ]; then
4  echo "You are an adult."
5  elif
6  echo "You are a minor."
7  else
8  echo "NO."
9  fi

```

Listing 3.6: if-else 语法

case 嵌套

```

1  case $TASK in
2      CB)
3          DEV=100
4          ;;
5      Copa)
6          DEV=100
7          TASK_ARGS="--train_as_classification False"
8          ;;
9      MultiRC)
10         GA=$(expr $BS / 2)
11         BS=2
12         TASK_ARGS="--gradient_accumulation_steps $GA"
13         ;;
14      ReCoRD)
15         GA=$(expr $BS / 2)
16         BS=2
17         TASK_ARGS="--gradient_accumulation_steps $GA --train_as_classification False"
18         ;;
19      *)
20         # 默认处理
21         ;;
22  esac

```

Listing 3.7: case 嵌套循环示例代码

for 循环

```

1  # 输出 1 到 5 的数字
2  for i in {1..5}; do

```

```

3     echo "Number_␣$i"
4 done

```

Listing 3.8: for-循环语法

while 循环

```

1  # 输出 1 到 5 的数字
2  i=1
3  while [ $i -le 5 ]; do
4      echo "Number_␣$i"
5      i=$((i + 1))
6  done

```

Listing 3.9: while-循环语法

until 循环

```

1  # 输出 1 到 5 的数字
2  i=1
3  until [ $i -gt 5 ]; do
4      echo "Number_␣$i"
5      i=$((i + 1))
6  done

```

Listing 3.10: until-循环语法

函数语法

```

1  # 定义函数
2  greet() {
3      echo "Hello,␣$1!"
4  }
5  # 调用函数
6  greet "Alice" # 输出: Hello, Alice!

```

Listing 3.11: 函数

文件操作

```

1  touch myfile.txt # 创建一个空文件
2  echo "This_␣is_␣a_␣test." > myfile.txt # 覆盖写入
3  echo "Another_␣line." >> myfile.txt # 追加写入
4  cat myfile.txt # 显示文件内容
5  rm myfile.txt # 删除文件
6  ...

```

Listing 3.12: 文件操作

shell-参数自动化调整**自动化调参 python-part**

```

1  import argparse
2  from transformers import HfArgumentParser
3  from dataclasses import dataclass
4

```

```

5  # 定义训练参数类
6  @dataclass
7  class OurArguments:
8      task_name: str = "SST2"
9      num_train: int = 0
10     model_name: str = "facebook/opt-125m"
11     # 其他参数...
12 # 方式1
13 def parse_args():
14     parser = argparse.ArgumentParser(description="Training script.")
15
16     # 添加你的参数
17     parser.add_argument("--task_name", type=str, default="SST2", help="Task name")
18     parser.add_argument("--num_train", type=int, default=0, help="Number of training samples")
19     parser.add_argument("--model_name", type=str, default="facebook/opt-125m", help="Model name")
20     # 可以继续添加其他参数
21     return parser.parse_args()
22 # 方式2[主要使用]
23 def parse_args():
24     parser = HfArgumentParser(OurArguments)
25     args = parser.parse_args_into_dataclasses()[0]
26     return args
27
28 def main():
29     # 解析命令行参数
30     args = parse_args()
31     # 使用参数
32     args.name = name
33
34 if __name__ == "__main__":
35     main()

```

Listing 3.13: 调参

自动化调参 shell-part

```

1  #!/bin/bash
2
3  # 定义任务、模型和训练样本数量的不同配置
4  TASKS=("SST2" "RTE" "CB")
5  MODEL_NAMES=("facebook/opt-125m" "google/bert-large-uncased")
6  NUM_TRAIN_VALUES=(100 200 300)
7
8  # 自动化调参
9  for TASK in "${TASKS[@]}"
10 do
11     for MODEL in "${MODEL_NAMES[@]}"
12     do
13         for NUM_TRAIN in "${NUM_TRAIN_VALUES[@]}"
14         do
15             echo "Running for Task: $TASK, Model: $MODEL, Num_train: $NUM_TRAIN"
16

```

```

17         # 执行 Python 脚本并传递参数
18         python train.py --task_name $TASK --model_name $MODEL --num_train
           $NUM_TRAIN
19     done
20 done
21 done

```

Listing 3.14: 调参

运行脚本

```

1  # Windows 版本(示例)
2  python -u "f:\C_Code_AI_Program\C_Generative\GAN1_Image_Style_Transfer\trainrun.py"
      --gpu_ids 0 --task_name apple2orange --lr 0.0002 --batch_size 1 --epochs 1000 --
      size 256 --decay_epoch 60
3  # Linux 版本(示例)
4  GPU_IDS=0 TASK_NAME=apple2orange LR=0.0002 BATCH_SIZE=1 EPOCHS=1000 SIZE=256
      DECAY_EPOCH=60 bash run.sh

```

Listing 3.15: 运行代码

3.1.3 进度条设置

进度条需要采用 `tqdm` 包，基本使用方法如下：基本用法

```

1  from tqdm import tqdm
2  import time
3
4  ## 1. 基本用法
5  # 普通循环（没有进度条）
6  # for i in range(100):
7  #     time.sleep(0.05) # 模拟耗时操作
8
9  # 用 tqdm 包装后（有进度条）
10 for i in tqdm(range(100), desc="Processing", unit="单位"):
11     time.sleep(0.05) # 模拟耗时操作
12
13 ## 2. 包装列表等可迭代对象
14 items = ["a", "b", "c", "d", "e"] * 20 # 构造一个列表
15
16 # 包装列表
17 for item in tqdm(items, desc="Processing_items"):
18     time.sleep(0.1) # 模拟处理每个元素的耗时
19
20 ## 3. 手动更新进度条
21 # 创建进度条对象，总步数为 3
22 pbar = tqdm(total=3, desc="手动更新")
23 # 第一步
24 time.sleep(1)
25 pbar.update(1) # 进度 +1
26 # 第二步
27 time.sleep(1)
28 pbar.update(1) # 进度 +1
29 # 第三步

```

```

30     time.sleep(1)
31     pbar.update(1) # 进度 +1
32     pbar.close() # 关闭进度条
33
34     ## 4. 在嵌套循环中使用
35     epochs = 3 # 总轮次
36     batchs_per_epoch = 10 # 每轮的批次
37
38     # 外层循环：总训练进度
39     global_pbar = tqdm(total=epochs * batchs_per_epoch, desc="总进度")
40
41     for epoch in range(epochs):
42         # 内层循环：当前epoch的进度 (leave=False 表示完成后清除)
43         epoch_pbar = tqdm(range(batchs_per_epoch),
44                             desc=f"第{epoch+1}轮",
45                             leave=False)
46
47         for batch in epoch_pbar:
48             time.sleep(0.2) # 模拟训练一个批次
49             global_pbar.update(1) # 总进度 +1
50             epoch_pbar.set_postfix(loss=0.5 - epoch*0.1) # 显示额外信息 (如损失)
51
52         epoch_pbar.close()
53
54     global_pbar.close()

```

Listing 3.16: 示例

3.1.4 查询服务器显卡属性设置

训练显存消耗记录

```

1     import torch
2
3     def track_gpu_memory(step_name):
4         # 当前已分配显存
5         current = torch.cuda.memory_allocated() / 1024**3
6         # 历史最大显存 (峰值)
7         peak = torch.cuda.max_memory_allocated() / 1024**3
8         # 重置峰值统计 (可选, 用于分阶段监控)
9         # torch.cuda.reset_peak_memory_stats()
10        print(f"[{step_name}] 显存: 当前{current:.2f}GB | 峰值{peak:.2f}GB")
11
12    # 训练循环中关键节点调用
13    for epoch in range(epochs):
14        for inputs, labels in dataloader:
15            # 前向传播前
16            track_gpu_memory("前向传播前")
17
18            # 前向传播
19            outputs = model(inputs)
20            loss = criterion(outputs, labels)
21            track_gpu_memory("前向传播后")

```

```

22
23     # 反向传播
24     optimizer.zero_grad()
25     loss.backward()
26     track_gpu_memory("反向传播后")
27
28     # 优化器更新
29     optimizer.step()
30     track_gpu_memory("优化器更新后")
31
32     # 清空当前批次变量
33     del inputs, labels, outputs, loss
34     torch.cuda.empty_cache() # 手动触发缓存清理 (可选)
35     track_gpu_memory("批次结束后")

```

Listing 3.17: 查询显存

训练时长消耗记录

```

1     import time
2     from datetime import timedelta
3
4     # 记录开始时间
5     start_time = time.time()
6
7     # 训练代码...
8     for epoch in range(epochs):
9         # 训练过程
10        pass
11
12    # 计算总时长
13    total_time = time.time() - start_time
14    print(f"总运行时间:{timedelta(seconds=int(total_time))}")

```

Listing 3.18: 训练时长记录

3.1.5 wandb 学习

演示示例如下:

```

1     import wandb
2     import random
3     import numpy as np
4     import os
5
6     def main():
7         # 1. wandb 登录过程
8         print("====_登录到Weights_&_Biases_====")
9         try:
10            # 尝试检查登录状态
11            api = wandb.Api()
12            print("已登录到wandb")
13        except Exception:
14            print("未检测到登录状态, 需要进行登录")

```



```

15     # 优先从环境变量获取API密钥（适合服务器环境）
16     api_key = os.getenv("WANDB_API_KEY")
17     if api_key:
18         wandb.login(key=api_key)
19         print("通过环境变量成功登录")
20     else:
21         # 交互式登录（适合本地开发）
22         print("请在弹出的浏览器中登录，或输入API密钥")
23         wandb.login()
24
25     # 2. 初始化wandb实验
26     print("\n====_初始化实验_====")
27     run = wandb.init(
28         project="wandb-complete-demo", # 项目名称
29         name=f"demo-run-{random.randint(1000, 9999)}", # 实验名称，包含随机数避免重
30             复
31         notes="这是一个wandb完整功能演示", # 实验备注
32         tags=["demo", "tutorial"], # 实验标签
33         config={ # 超参数配置
34             "learning_rate": 0.01,
35             "epochs": 20,
36             "batch_size": 32,
37             "optimizer": "Adam",
38             "dataset": "synthetic"
39         }
40     )
41
42     # 获取配置
43     config = wandb.config
44
45     # 3. 模拟训练数据
46     print("\n====_开始训练_====")
47     # 生成模拟数据分布的参数
48     true_mean = 0.5
49     true_std = 0.2
50
51     # 4. 训练循环
52     for epoch in range(config.epochs):
53         # 模拟训练过程中的指标
54         # 随着epoch增加，损失应该下降，准确率应该上升
55         loss = (true_mean + np.random.normal(0, true_std)) / (epoch + 1)
56         accuracy = 0.5 + (epoch / config.epochs) * 0.4 + np.random.normal(0, 0.03)
57         accuracy = min(accuracy, 0.98) # 限制最大准确率
58
59         # 记录指标到wandb
60         wandb.log({
61             "training/loss": loss,
62             "training/accuracy": accuracy,
63             "training/epoch": epoch,
64             # 可以记录更多衍生指标
65             "training/loss_derivative": loss * 0.1 if epoch > 0 else 0
66         })

```

```

67     # 打印进度
68     if epoch % 5 == 0: # 每5个epoch打印一次
69         print(f"Epoch_{epoch:2d}/{config.epochs}:_{"
70             f"Loss_{loss:.4f},_"
71             f"Accuracy_{accuracy:.4f}")
72
73     # 5. 完成实验
74     print("\n====_训练完成_====")
75     wandb.finish()
76     print(f"实验已成功记录，可在以下地址查看：")
77     print(f"https://wandb.ai/{wandb.run.entity}/{config.project}/runs/{wandb.run.id}"
78           )
79
80 if __name__ == "__main__":
81     main()

```

Listing 3.19: wandb 记录

3.1.6 Github 项目管理

3.1.7 Tikz 的使用查询

基础环境

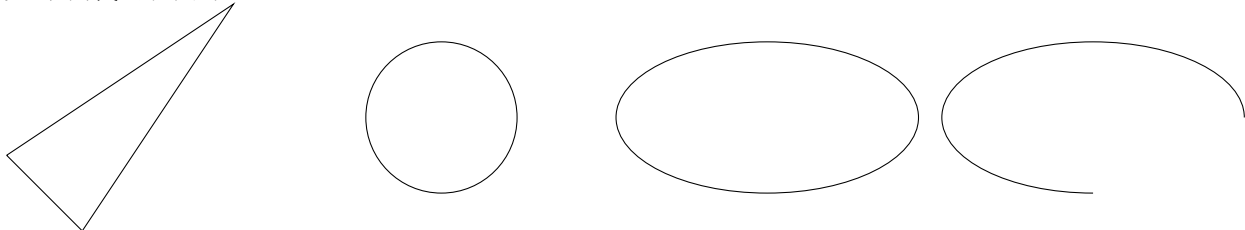
```

1 \usepackage{tikz}
2 \begin{tikzpicture}
3     ...
4 \end{tikzpicture}

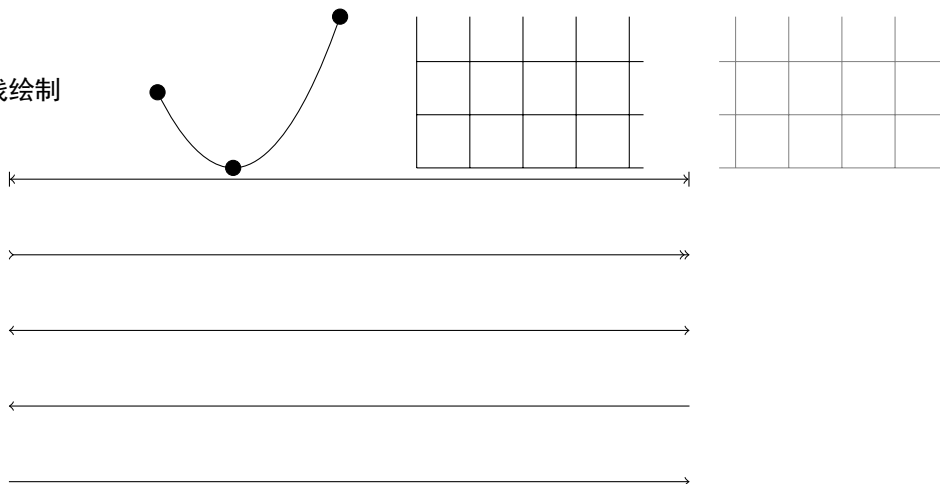
```

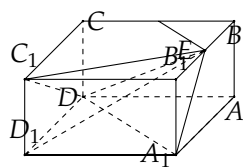
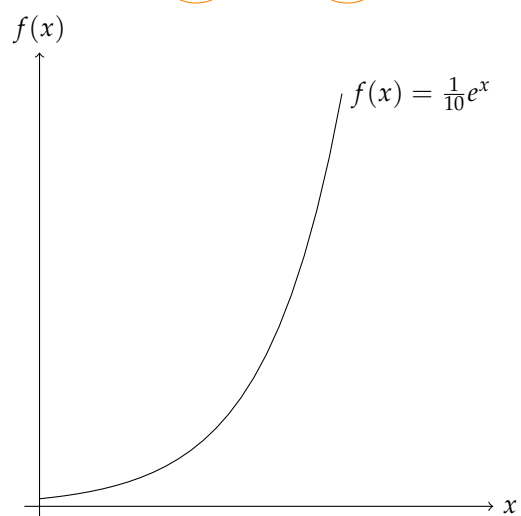
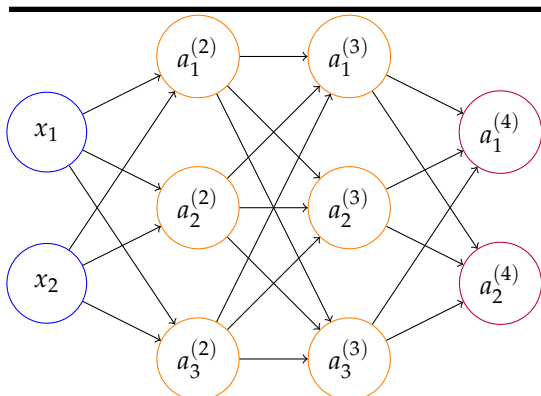
Listing 3.20: tikz 基础

图形绘制 示例代码和图片：



曲线绘制





3.2.1 基本流程

通常深度学习领域，我们将

1. 数据准备

1.1 数据收集

- 下载、抓取或者合成数据
- 公开数据集（ImageNet、COCO、MNIST 等）
- 定义数据集（遥感图像、医学图像、草图数据等）
- 数据增强（旋转、翻转、模糊等）

1.2 数据预处理

- 归一化
- 格式转换
- 数据储存形式等

1.3 数据加载

- 定义批处理

注：数据可以进行动态加载，无需下载到本地

2. 定义模型

- 选择合适的架构
- 决定模型的层数，参数量
- 加载预训练模型

3. 定义损失函数和优化器

3.1 选择损失函数

- 分类任务：交叉熵，折页损失函数 (SVM)
- 回归任务：均方差 (MSE)
- 对象检测或者分割：交并比 (ioU)
- 策略优化：KL 散度
- 词嵌入：噪音对比估计 (NCE)
- 词向量：余弦相似度
- 等等

3.2 选择优化器

- SGD
- AdamW

3.3 学习率的调度

- StepLR
- Cosine Annealing

4. 训练模型在每个 epoch 中

训练阶段

- 前向传播
- 计算损失
- 反向传播

验证阶段

- 计算在验证集上的损失
- 评估指标

5. 评估和测试

- 计算指标
- 绘制 loss 曲线
- 生成可视化结果

6. 模型保存和记录

6.1 保存最佳模型

6.2 记录训练信息

- 日志信息
- wandb
- 记录训练时长
- 记录模型的显存峰值
- 等等

7. 部署和推理

- 导出 ONNX/TensorRT python
- 加载模型进行推理

8. 超参数调优

3.2.2 基本代码模块详解

A. 数据准备

在数据处理、编程开发或机器学习等领域，数据加载（Data Loading）是指将数据从存储位置（如文件、数据库、网络接口等）读取到程序的内存中，使其成为可被程序直接访问和处理的格式的过程。它是数据处理流程中的基础环节，为后续的清洗、分析、建模等操作提供数据支持。通常采用：

- RGB 数据、OR BGR 数据
- JPEG 编码后的数据
- torchvision.datasets 中 shuju
- torch.utils.data 下的 Dataset, Dataloader 自定义的数据集

在机器学习（尤其是计算机视觉、自然语言处理等领域）中，数据增强（Data Augmentation）是一种通过对原始训练数据进行合理的、有策略的变换或扩展，生成新的“虚拟样本”，从而扩大训练数据集规模、丰富数据多样性的技术。其核心目标是：让模型在训练时接触到更多样化的输入，增强模型对数据中各种变化（如噪声、变形、视角差异等）的鲁棒性，减少过拟合（即模型过度依赖训练数据的细节，在新数据上表现不佳）。通常采用：

```

1  from torchvision import transforms
2
3  # 定义数据增强
4  train_transform = transforms.Compose([
5      # 随机剪切
6      transforms.RandomResizedCrop((28, 28)),
7      # 随机水平翻转
8      transforms.RandomHorizontalFlip(),
9      # 随机垂直翻转
10     transforms.RandomVerticalFlip(),
11     # 随机旋转
12     transforms.RandomRotation(90),
13     # 随机转换为灰度图
14     transforms.RandomGrayscale(p=0.1),
15     # 颜色信息增强
16     transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.3),
17     # 转换为Tensor
18     transforms.ToTensor(),
19     # 归一化
20     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
21     # 等等
22 ])

```

Listing 3.21: 数据增强格式

对于测试集，并不需要一些变换，仅需定义 transform 将数据转换为 Tensor 即可。

B. 定义模型

如果需要自己手动搭建模型，那么常见的模型文件模块如下：

例子 1: 手动搭建 VGG 串联模型

```

1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4
5  class VGGbase(nn.Module):
6      def __init__(self):
7          super(VGGbase, self).__init__()
8
9      # 构建一个神经网络模型的容器用于封装多个神经网络层
10     # -----
11     self.conv1 = nn.Sequential(
12         nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1),
13         nn.BatchNorm2d(64),
14         nn.ReLU(inplace=False)
15     )
16
17     # 池化操作，保留特征的同时减少尺寸
18     self.max_pooling1 = nn.MaxPool2d(kernel_size=2, stride=2)
19     # -----
20     self.conv2_1 = nn.Sequential(
21         nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1),

```

```

22     nn.BatchNorm2d(128),
23     nn.ReLU(inplace=False)
24 )
25
26     self.conv2_2 = nn.Sequential(
27         nn.Conv2d(128, 128, kernel_size=3, stride=1, padding=1),
28         nn.BatchNorm2d(128),
29         nn.ReLU(inplace=False)
30     )
31     # 池化操作, 保留特征的同时减少尺寸
32     self.max_pooling2 = nn.MaxPool2d(kernel_size=2, stride=2)
33     # -----
34     self.conv3_1 = nn.Sequential(
35         nn.Conv2d(128, 256, kernel_size=3, stride=1, padding=1),
36         nn.BatchNorm2d(256),
37         nn.ReLU(inplace=False)
38     )
39
40     self.conv3_2 = nn.Sequential(
41         nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1),
42         nn.BatchNorm2d(256),
43         nn.ReLU(inplace=False)
44     )
45
46     # 池化操作, 保留特征的同时减少尺寸
47     self.max_pooling3 = nn.MaxPool2d(kernel_size=2, stride=2, padding=1)
48     # -----
49     self.conv4_1 = nn.Sequential(
50         nn.Conv2d(256, 512, kernel_size=3, stride=1, padding=1),
51         nn.BatchNorm2d(512),
52         nn.ReLU(inplace=False)
53     )
54
55     self.conv4_2 = nn.Sequential(
56         nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1),
57         nn.BatchNorm2d(512),
58         nn.ReLU(inplace=False)
59     )
60     # 池化操作, 保留特征的同时减少尺寸
61     self.max_pooling4 = nn.MaxPool2d(kernel_size=2, stride=2)
62
63     # batchsize * 512 * 2 * 2
64     self.fc = nn.Linear(512 * 4, 10)
65
66     def forward(self, x):
67         batchsize = x.size(0)
68         out = self.conv1(x)
69         out = self.max_pooling1(out)
70
71         out = self.conv2_1(out)
72         out = self.conv2_2(out)
73         out = self.max_pooling2(out)
74

```

```

75     out = self.conv3_1(out)
76     out = self.conv3_2(out)
77     out = self.max_pooling3(out)
78
79     out = self.conv4_1(out)
80     out = self.conv4_2(out)
81     out = self.max_pooling4(out)
82
83     out = out.view(batchsize, -1)
84     out = self.fc(out)
85     out = F.log_softmax(out, dim=1)
86
87     return out
88
89 def VGGNet():
90     return VGGbase()

```

Listing 3.22: VGG 串联模型搭建示例

串联模型的好处就是简便，可以将想加入层按顺序放在模型中，自行增加模型的深度。

例子 2: 手动搭建 ResNet 跳联模型

```

1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4
5  # ResNet 的模块
6  class ResBlock(nn.Module):
7      def __init__(self, in_channels, out_channels, stride=1):
8          super(ResBlock, self).__init__()
9
10         # 主干层
11         self.layer = nn.Sequential(
12             nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1),
13             nn.BatchNorm2d(out_channels),
14             nn.ReLU(),
15             nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1),
16             nn.BatchNorm2d(out_channels)
17         )
18
19         # 跳跃连接
20         self.shortcut = nn.Sequential()
21         if in_channels != out_channels or stride != 1:
22             self.shortcut = nn.Sequential(
23                 nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride),
24                 nn.BatchNorm2d(out_channels)
25             )
26
27         def forward(self, x):
28             out1 = self.layer(x)
29             out2 = self.shortcut(x)
30             out = out1 + out2
31             out = F.relu(out)
32             return out
33
34 class ResNet_base(nn.Module):

```



```

33
34     def make_layer(self, block, out_channels, stride=1, num_blocks=1):
35         layers_list = []
36         for i in range(num_blocks):
37             if i == 0:
38                 in_stride = stride
39             else:
40                 in_stride = 1
41             layers_list.append(block(self.in_channels, out_channels, in_stride))
42             self.in_channels = out_channels
43         return nn.Sequential(*layers_list)
44
45     def __init__(self):
46         super(ResNet_base, self).__init__()
47         self.in_channels = 32
48         self.conv1 = nn.Sequential(
49             nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1),
50             nn.BatchNorm2d(32),
51             nn.ReLU(inplace=False)
52         )
53         self.layer1 = self.make_layer(ResBlock, 64, stride=2, num_blocks=3)
54         self.layer2 = self.make_layer(ResBlock, 128, stride=2, num_blocks=3)
55         self.layer3 = self.make_layer(ResBlock, 256, stride=2, num_blocks=3)
56         self.layer4 = self.make_layer(ResBlock, 512, stride=2, num_blocks=3)
57         self.fc = nn.Linear(512, 10)
58
59     def forward(self, x):
60         out = self.conv1(x)
61         out = self.layer1(out)
62         out = self.layer2(out)
63         out = self.layer3(out)
64         out = self.layer4(out)
65         out = F.avg_pool2d(out, 2)
66         out = out.view(out.size(0), -1)
67         out = self.fc(out)
68         return out
69
70     def ResNet():
71         return ResNet_base()

```

C. 定义损失函数和优化器

D. 训练和测试模型

E. 训练和测试

3.2.3 定制代码模块详解

3.2.4 基础环境配置

将代码上传到 linux 服务器之后，需要对环境进行初步的配置，后续需要什么补充什么即可。

```
1  sudo apt update && sudo apt install screen tree vim git htop gcc g++ colordiff python  
   -is-python3 jq -y  
2  mkdir -p ~/miniconda3  
3  wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86\_64.sh -O  
4  ~/miniconda3/miniconda.sh  
5  bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3  
6  rm ~/miniconda3/miniconda.sh  
7  source ~/miniconda3/bin/activate  
8  conda init --all  
9  conda update -n base -c defaults conda  
10 conda create -y -n envname python=3.9.7
```

Listing 3.23: 基础环境的配置

注：第 28 行可以合在一起复制给 (Linux) 终端。

3.3.1 图像识别 (Computer Vision)

3.3.2 自然语言处理 (Natural Language Processing)

情感分析

帮助用户快速获取、整理和分析相关的评价体系，对带有感情色彩的主观性文本进行、处理、归纳和推理。

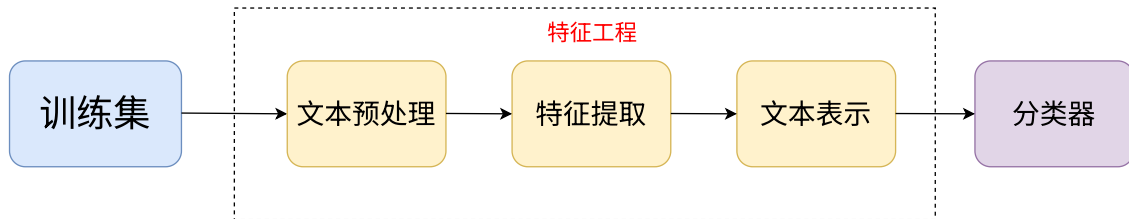


图 3.1: 情感分析基本流程

3.3.3 生成式学习 (Generative Learning)

3.3.4 多模态 (Multimodal)

3.3.5 脉冲神经网络相关 (SNN)

Chapter 4

第四章：AI 知识整理

Chapter 5

第五章：文章汇总整理

§5.1
黑盒-零阶优化问题

§5.2
大模型微调相关

§5.3
SNN 相关 [校准]

§5.4
SNN 相关 [量化]

§5.5
SNN 相关 [马尔可夫]

§5.6
SNN 相关 [编码]

Chapter 6

第六章：技术汇总整理

§6.1 梯度优化

6.1.1 神经网络架构搜索

参考文章：

DARTS: <https://arxiv.org/pdf/1806.09055>

相应背景知识：

神经网络架构搜索 (Neural Architecture Search)[NAS], 旨在找到特定任务 (如图像识别、语音识别) 最优的神经网络架构。NAS 属于自动机器学习 (AutoML) 的范畴。目标是减少甚至消除需要人工设计网络结构的需求。原理大致如下：

1. 搜索空间：定义可能的网络结构选项，包括各种层类型 (如卷积层，池化层，全连接层等)，层的数量，连接模式和激活函数等。
2. 搜索策略：通过某种算法在搜索空间中寻找最优结构 (进化算法，基于强化学习的算法，贝叶斯优化，梯度法等)。
3. 性能评估：对搜索到的网络结构进行训练并评估其性能 (准确率，推理时间，模型大小等度量标准)。

文章主要内容：

DARTS [1] 是在 ICLR 2019 上提出的。大多数 NAS 算法是使用 RL 算法或 Evolution 算法进行结构搜索，把它们当做 black box 优化问题。然而这样的话的 search space 是离散的，搜索非常耗时了。DARTS 试图解决这个问题，提出了一种新的网络搜索算法，它的 search space 是连续的，在搜索过程中它通过在验证集上验证，然后使用梯度下降算法来完成搜索。实验也表明了 DARTS 进行网络架构搜索的计算开销要比普通网络架构算法小几个量级，但搜索得到的网络结构却仍能 and 之前 SOTA 算法几乎持平。同时它的泛化能力也很好，不仅可以用于 CNN 结构的搜索，还可以用于 RNN 结构的搜索。

核心目标：在保证模型性能的同时，尽可能的降低搜索过程中的成本，同时提升架构的通用性。

具体方法：

网络架构搜索算法通常需明确三个核心部分：[搜索空间](#)、[搜索策略](#)、[性能评估](#)。

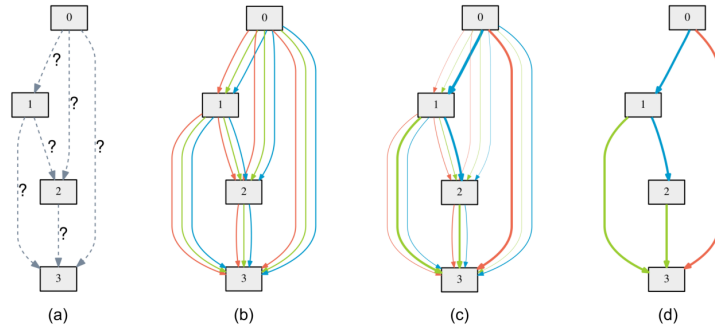
1.Search Space: DARTS 中搜索的最小单位也叫 cell，它是一个 DAG，以它为 block 来构建 architecture。每个 cell 以 N 个有序的 nodes 组成，每个 node $x^{(i)}$ 代表一个 latent representation，每条 edge (i, j) 代表在 $x^{(i)}$ 执行了某种 operation $o^{(i,j)}$ 。

$$x^{(j)} = \sum_{i < j} o^{(i,j)}(x^{(i)}). \quad (6.1)$$

2.Continuous Relaxation and Optimization: 我们用 $\mathcal{O}$ 来表示 candidate operations，到目前为止，search space 还是离散的。所以为了使 search space 连续，作者为 edge 上的每一种 operation 赋上一个 weight $\alpha_0^{(i,j)}$ 。所以 operation 可表示为：

$$\hat{o}^{(i,j)}(x) = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'}^{(i,j)})} o(x) \quad (6.2)$$

其实这也很简单，就是利用 $\alpha = \alpha_o^{(i,j)}$ 做一个 softmax 激活，然后对各种 operation 作加权平均罢了。那么问题实际上就是被转移成 learn 一组参数 $\alpha = \{\alpha_o^{(i,j)}\}$ 了，搜索结束后只要通过 $o^{(i,j)} = \arg \max_{o \in \mathcal{O}} \alpha_o^{(i,j)}$ 把 $\hat{o}^{(i,j)}$ 替换成 $o^{(i,j)}$ 就可以了。下图是一个示意图：



上述内容主要围绕神经网络架构优化展开，涉及以下核心内容：

1. 架构优化中，将可训练权重设为 w ，优化目标是最小化验证集损失 $\mathcal{L}_{val}(w^*, \alpha^*)$ 。当架构与 α^* 固定时，需通过 $w^* = \arg \min_w \mathcal{L}_{train}(w, \alpha^*)$ 确定最优权重 w^* 。
2. 利用参数 $\alpha = \alpha_o^{(i,j)}$ 进行 softmax 激活，对各类操作（operation）做加权平均，即：

$$\hat{o}^{(i,j)}(x) = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'}^{(i,j)})} o(x)$$

问题转化为学习参数集 $\alpha = \{\alpha_o^{(i,j)}\}$ ，搜索结束后通过 $o^{(i,j)} = \arg \max_{o \in \mathcal{O}} \alpha_o^{(i,j)}$ 替换得到最终操作 $o^{(i,j)}$ 。

3. 该思路属于神经网络架构搜索（NAS）范畴，通过分步优化架构参数 α^* 和权重参数 w^* ，实现神经网络架构的自动化设计。

Chapter 7

第七章：项目练习

参考文献

- [1] Hanxiao Liu, Karen Simonyan, and Yiming Yang. Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055*, 2018.

Chapter A

常见的的数据集汇总

§ A.1 图像数据

A.1.1 常见的 CycleGAN 数据:

官方链接:<https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/>

horse2zebra

数据来源: 伯克利官方提供的原始数据集, 包含 939 张马和 1177 张斑马的图片。

数据下载链接: <https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/horse2zebra.zip>

apple2orange

数据来源: 伯克利官方提供的原始数据集, 包含 996 张苹果和 1020 张橙子的图片。

数据下载链接: <https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/apple2orange.zip>

summer2winter_yosemite

数据来源: 伯克利官方提供的原始数据集, 包含 1273 张夏季和 854 张冬季约塞米蒂国家公园的图片。

数据下载链接: https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/summer2winter_yosemite.zip

monet2photo

数据来源: 伯克利官方提供的原始数据集, 包含 1074 张莫奈画作和 6853 张现实照片。

数据下载链接: <https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/monet2photo.zip>

cezanne2photo

数据来源: 伯克利官方提供的原始数据集, 包含 584 张塞尚画作和 6853 张现实照片。

数据下载链接: <https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/cezanne2photo.zip>

ukiyoe2photo

数据来源：伯克利官方提供的原始数据集，包含 1433 张浮世绘画作和 6853 张现实照片

数据下载链接：<https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/ukiyoe2photo.zip>

vangogh2photo

数据来源：伯克利官方提供的原始数据集，包含 401 张梵高画作和 6853 张现实照片。

数据下载链接：<https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/vangogh2photo.zip>

maps

数据来源：伯克利官方提供的原始数据集，包含 1096 张地图和相应的卫星图像。

数据下载链接：<https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/maps.zip>

facades

数据来源：伯克利官方提供的原始数据集，包含 400 张建筑立面的图片。

数据下载链接：<https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/facades.zip>

iphone2dslr_flower

数据来源：伯克利官方提供的原始数据集，包含 1813 张 iPhone 拍摄的花卉图片和 3316 张 DSLR 拍摄的花卉图片。

数据下载链接：https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/iphone2dslr_flower.zip

ae_photos

数据来源：伯克利官方提供的原始数据集，包含 1000 张 AE 风格的图片和 1000 张现实照片。

数据下载链接：https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/ae_photos.zip

§ A.2 文本数据

§ A.3 脉冲数据

§ A.4 音频数据

§ A.5 视频数据